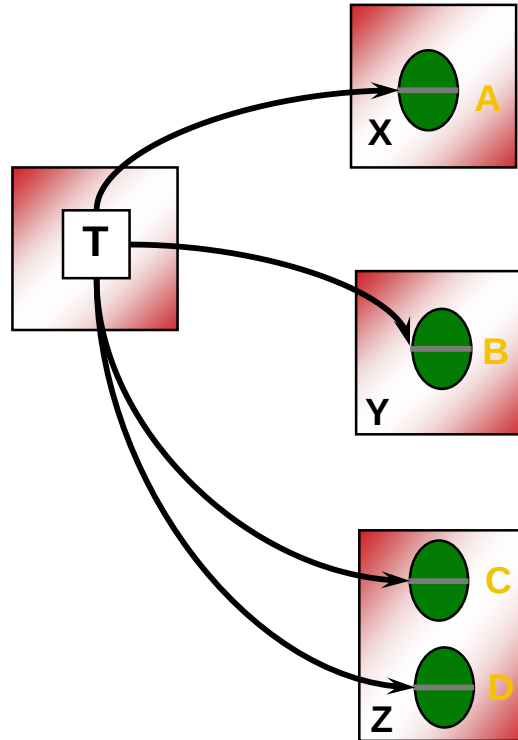


Commit Protocols

Edited slides of
Pallabh Dasgupta IITKgp
D Goswami IIT Guwahati

Distributed Transactions

❖ A transaction that invokes operations at several servers.



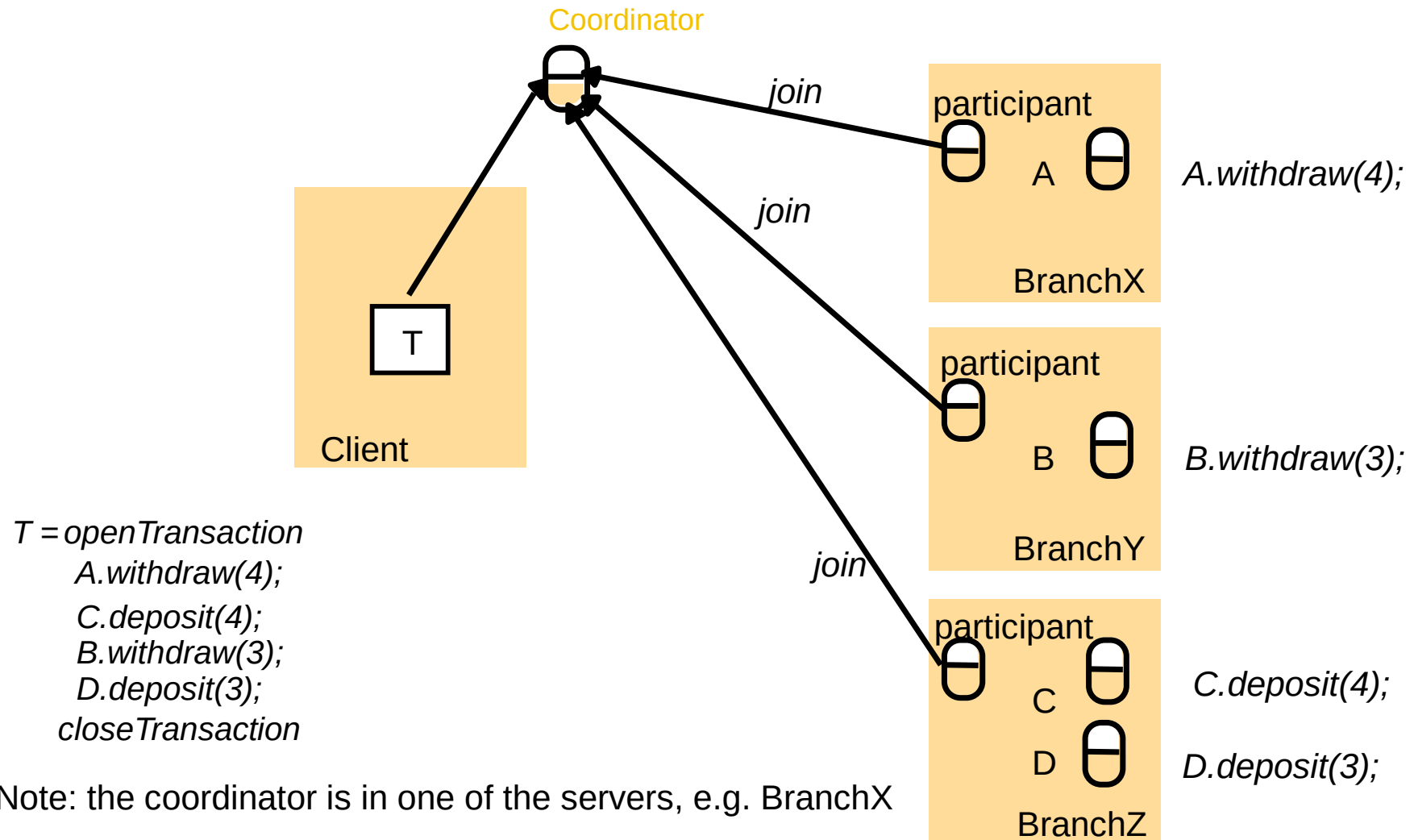
Coordinator of a Distributed Transaction

In a distributed environment, a coordinator is needed

Client sends an *openTransaction* to the coordinator

- Other servers that manage the objects accessed by the transaction become *participants*.

Distributed banking transaction



Transaction needs to follow the ACID properties

Atomicity—all or none—Transaction is an atomic unit. It is performed either to entirety or not performed at all.

Consistency—don't violate DB integrity constraints: execution of the operation should be correct taking from one consistent state to another.
Example: Amount can't be negative, total amount in the bank remains the same after within bank transfer, etc.

Isolation (Atomicity)—partial results are hidden. That is, execution of a transaction should not be interfered with by other transactions executing concurrently.

Durability—effects (of transactions that "happened" or committed) are forever. They should not be lost due to failure

Two Phase Commit Protocol (2PC)

- **Acquires locks before accessing any record**
- **Lock released only after transaction is either committed or aborted**
- **This can land in deadlock situations**
- **Why do we need to keep the locks?**

System Failure Modes

- Failures unique to distributed systems:
 - Failure of a site.
 - Loss of messages
 - Handled by network transmission control protocols such as TCP-IP
 - Failure of a communication link
 - Handled by network protocols, by routing messages via alternative links
 - Network partition
 - A network is said to be partitioned when it has been split into two or more subsystems that lack any connection between them
 - Note: a subsystem may consist of a single node
- Network partitioning and site failures are generally indistinguishable.
- Site A commits as it completes its work but site B realizes there is an error so has to abort – violates atomicity

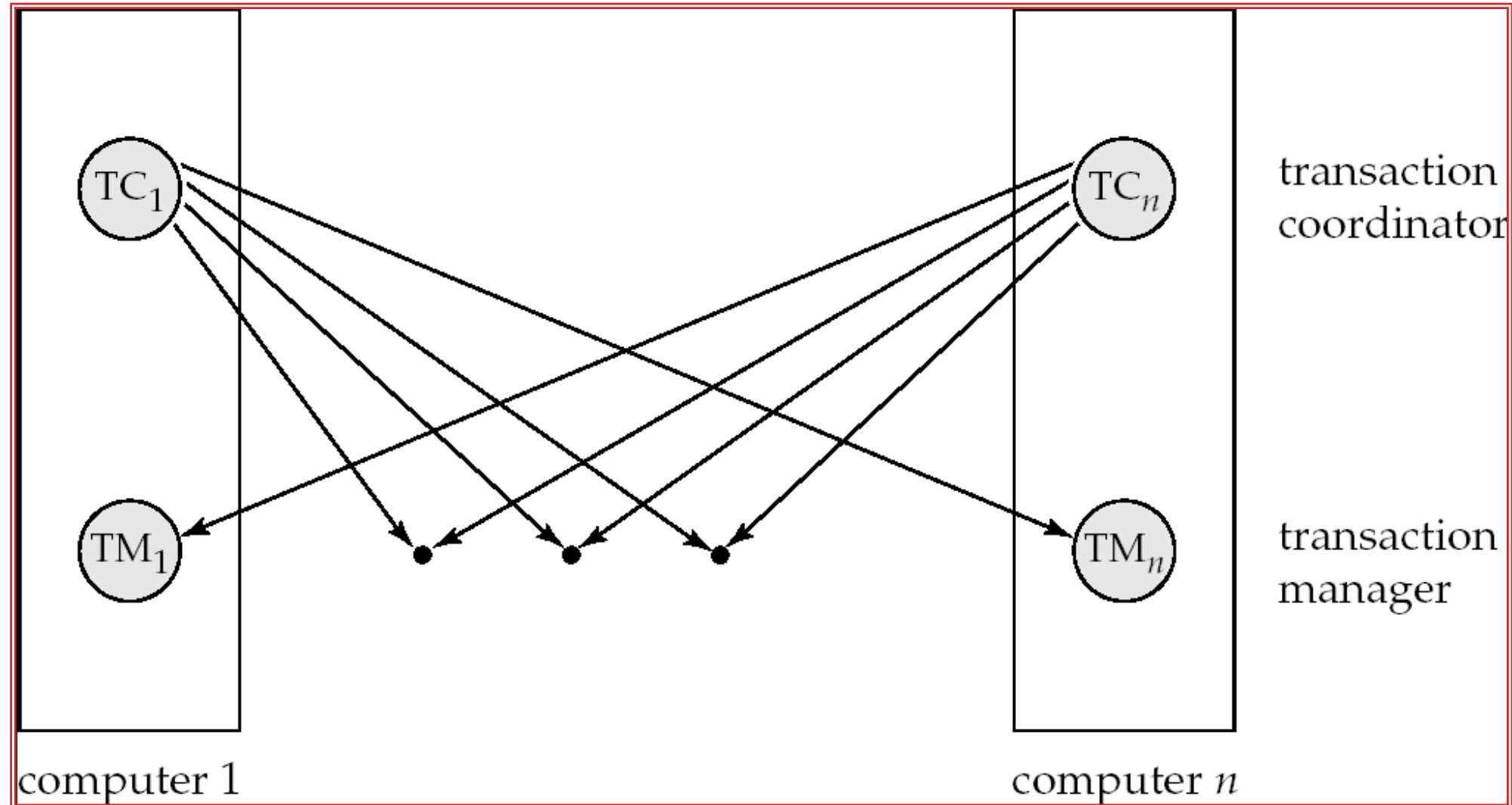
Commit Protocols

- Commit protocols are used to ensure atomicity across sites
 - a transaction which executes at multiple sites must either be committed at all the sites, or aborted at all the sites.
 - not acceptable to have a transaction committed at one site and aborted at another
- The *two-phase commit* (2PC) protocol is widely used
- The *three-phase commit* (3PC) protocol is more complicated and more expensive, but avoids some drawbacks of two-phase commit protocol.

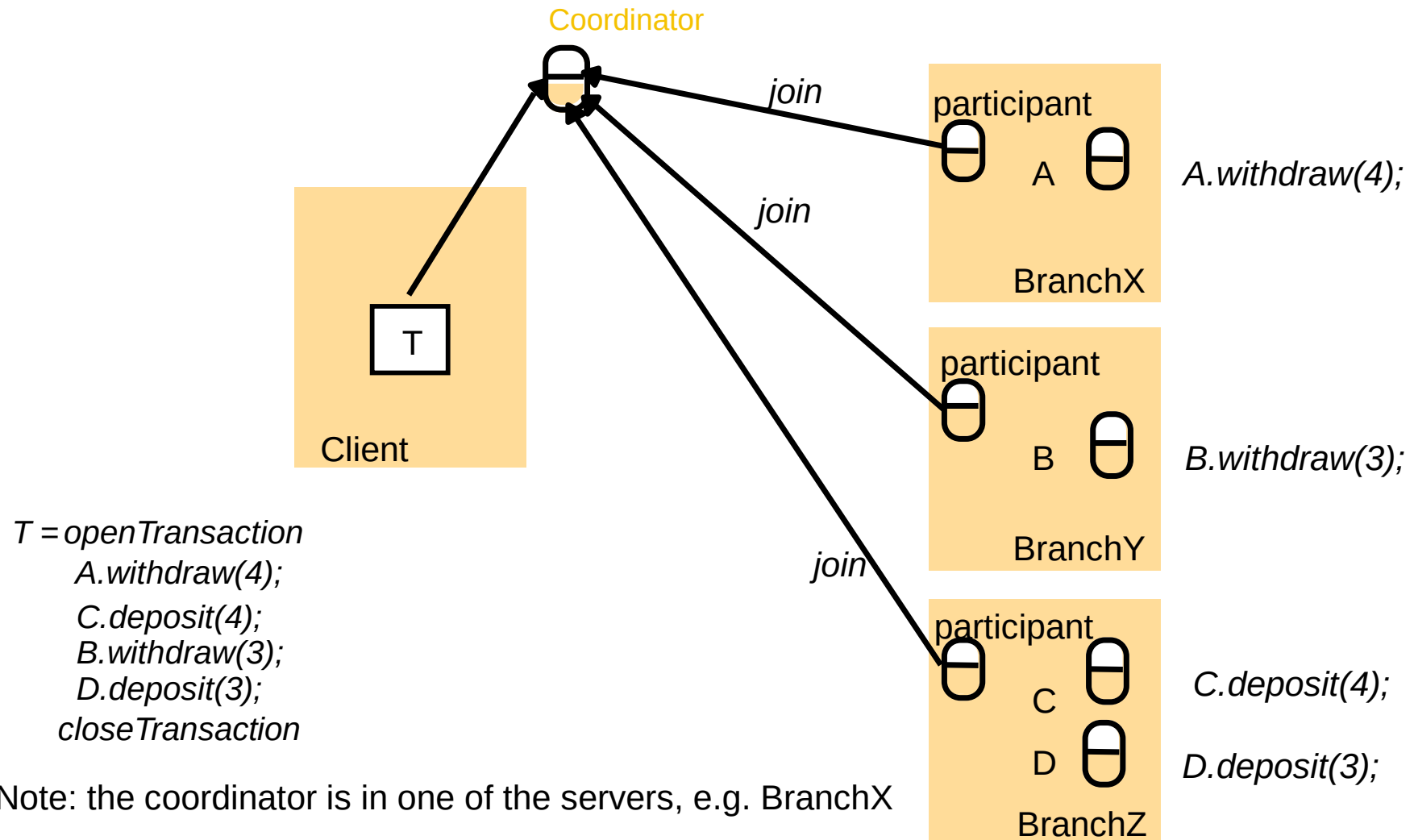
Distributed Transactions

- Transaction may access data at several sites.
- Each site has a local **transaction manager** responsible for:
 - Maintaining a log for recovery purposes
 - Participating in coordinating the concurrent execution of the transactions executing at that site.
- Each site has a **transaction coordinator**, which is responsible for:
 - Starting the execution of transactions that originate at the site.
 - Distributing subtransactions at appropriate sites for execution.
 - Coordinating the termination of each transaction that originates at the site, which may result in the transaction being committed at all sites or aborted at all sites.

Transaction System Architecture



Distributed banking transaction



Two Phase Commit Protocol (2PC)

- Assumes **fail-stop** model – failed sites simply stop working, and do not cause any other harm, such as sending incorrect messages to other sites. They can come up later
- Execution of the protocol is initiated by the coordinator after the last step of the transaction has been reached.
- The protocol involves all the local sites at which the transaction executed
- Let T be a transaction initiated at site S_i , and let the transaction coordinator at S_i be C_i

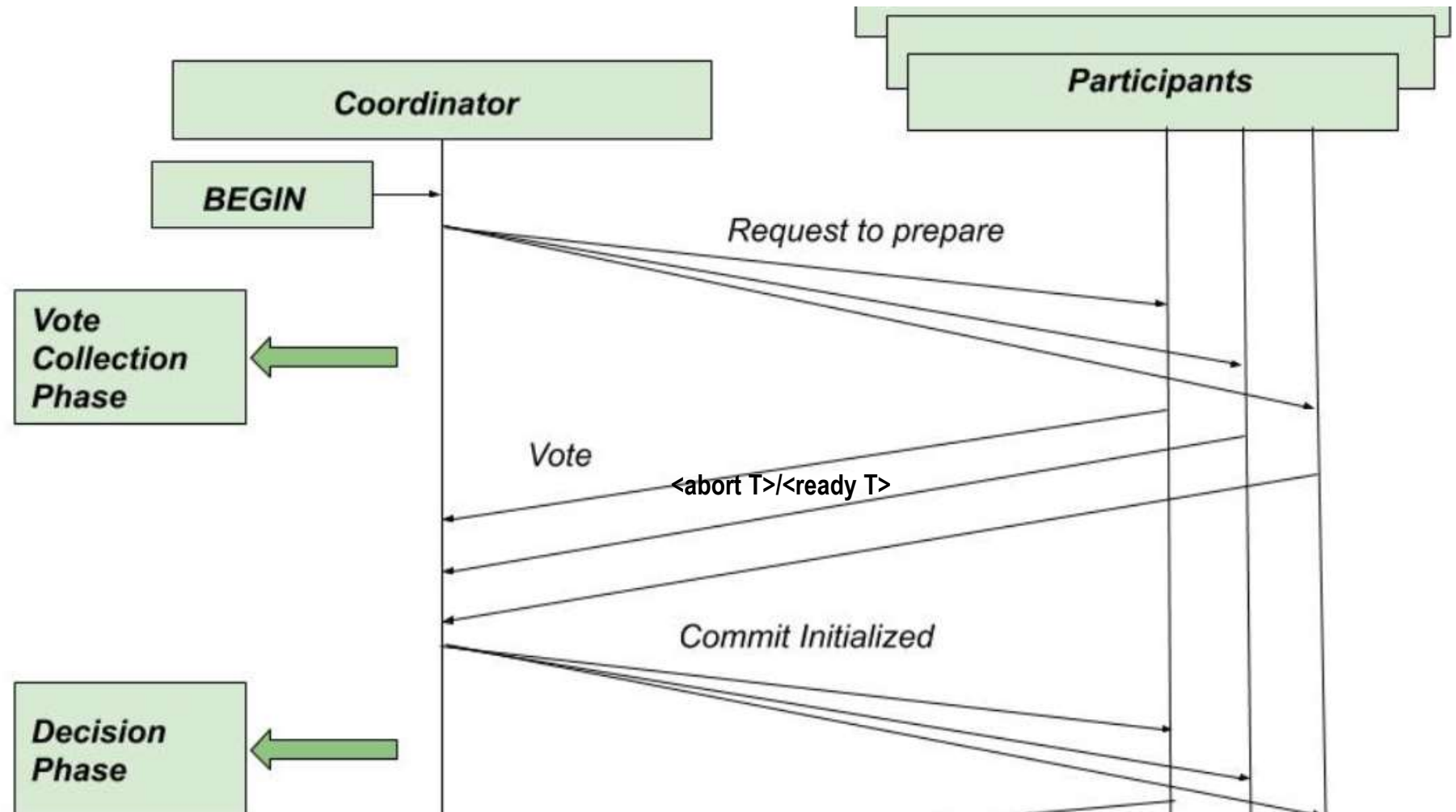
Phase 1: Obtaining a Decision

- Coordinator asks all participants to *prepare* to commit transaction T_i .
 - C_i adds the records $\langle \text{prepare } T \rangle$ to the log and forces log to stable storage
 - sends prepare T messages to all sites at which T executed
- Upon receiving message, transaction manager at site determines if it can commit the transaction
 - if not, add a record $\langle \text{no } T \rangle$ to the log and send abort T message to C_i
 - if the transaction can be committed, then:
 - add the record $\langle \text{ready } T \rangle$ to the log
 - force *all records* for T to stable storage
 - send ready T message to C_i

** Hence prepare $\langle T \rangle$ and $\langle \text{no } T \rangle$ or $\langle \text{ready } T \rangle$ are now stored in log which is now in stable storage (database untouched)*

Phase 2: Recording the Decision

- T can be committed if C_i received a ready T message from all the participating sites: otherwise T must be aborted.
- Coordinator adds a decision record, $\langle \text{commit } T \rangle$ or $\langle \text{abort } T \rangle$, to the log and forces record onto stable storage. Once the record stable storage it is irrevocable (even if failures occur)
- Coordinator sends a message to each participant informing it of the decision (commit or abort)
- Participants take appropriate action locally.



Handling of Failures - Site Failure

When site S_i recovers, it examines its log to determine the fate of transactions active at the time of the failure.

- Log contain $\langle \text{commit } T \rangle$ record: site executes redo (T) **copies from log to db*
- Log contains $\langle \text{abort } T \rangle$ record: site executes undo (T) **remove from log*
- Log contains $\langle \text{ready } T \rangle$ record: site must consult C_i to determine the fate of T .
 - If T committed, redo (T)
 - If T aborted, undo (T)
- The log contains no control records concerning T implies that S_k failed before responding to the prepare T message from C_i
 - since the failure of S_k precludes the sending of such a response C_i must abort T
 - S_k must execute undo (T)

Handling of Failures- Coordinator Failure

- If coordinator fails while the commit protocol for T is executing then participating sites must decide on T 's fate:
 1. If an active site contains a $\langle \text{commit } T \rangle$ record in its log, then T must be committed.
 2. If an active site contains an $\langle \text{abort } T \rangle$ record in its log, then T must be aborted.
 3. If some active participating site does not contain a $\langle \text{ready } T \rangle$ record in its log, then the failed coordinator C_i cannot have decided to commit T . Can therefore abort T .
 4. If none of the above cases hold, then all active sites must have a $\langle \text{ready } T \rangle$ record in their logs, but no additional control records (such as $\langle \text{abort } T \rangle$ or $\langle \text{commit } T \rangle$). In this case active sites must wait for C_i to recover, to find decision.

** we don't know C_i 's local decision as a participating site. Also we don't know if all ready T reached. If has not reached then C_i has not written in db so we don't need to commit.
- **Blocking problem** : active sites may have to wait for failed coordinator to recover (hence resources locked and held up). Again Coordinator failure very uncommon

Handling of Failures - Network Partition

- If the coordinator and all its participants remain in one partition, the failure has no effect on the commit protocol.
- If the coordinator and its participants belong to several partitions:
 - *Sites that are not in the partition containing the coordinator think the coordinator has failed, and execute the protocol to deal with failure of the coordinator.
 - No harm results, but sites may still have to wait for decision from coordinator.
 - *The coordinator and the sites which are in the same partition as the coordinator think that the sites in the other partition have failed, and follow the usual commit protocol.
 - Again, no harm results

Recovery and Concurrency Control

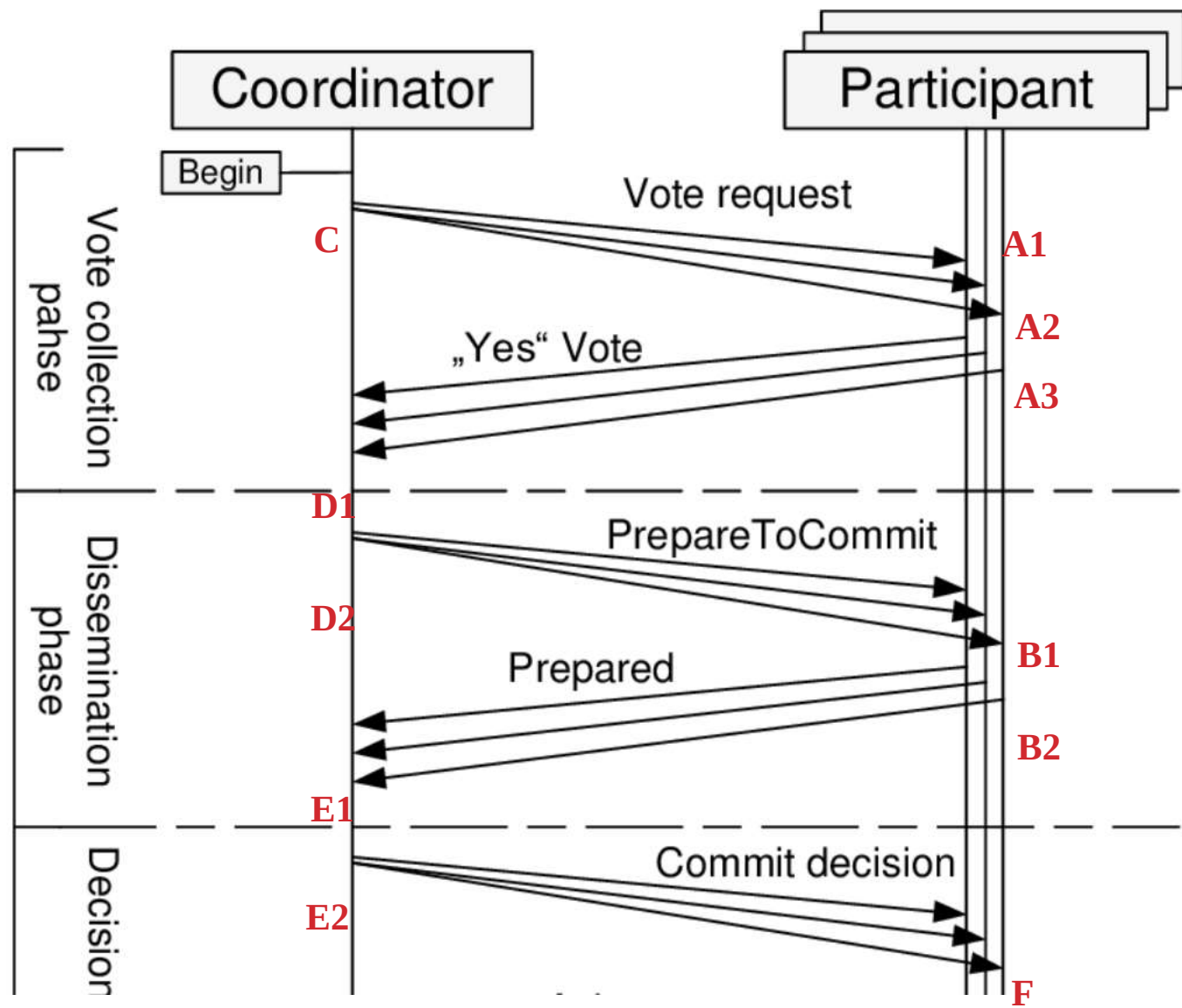
- **In-doubt transactions** have a $\langle \text{ready } T \rangle$, but neither a $\langle \text{commit } T \rangle$, nor an $\langle \text{abort } T \rangle$ log record.
- The recovering site must determine the commit-abort status of such transactions by contacting other sites; this can slow and potentially block recovery.
- Recovery algorithms can note lock information in the log.
 - Instead of $\langle \text{ready } T \rangle$, write out $\langle \text{ready } T, L \rangle$ L = list of locks held by T when the log is written (read locks can be omitted).
 - For every in-doubt transaction T , all the locks noted in the $\langle \text{ready } T, L \rangle$ log record are reacquired.
- After lock reacquisition, transaction processing can resume; the commit or rollback of in-doubt transactions is performed concurrently with the execution of new transactions.

Three Phase Commit (3PC)

- **Assumptions:**
 - No network partitioning
 - Bounded crash recovery/message delay/processing time.
 - Fail stop crash (node can recover and not malicious)
 - At any point, at least one site must be up.
- **Phase 1: Obtaining Preliminary Decision: Identical to 2PC Phase 1.**
 - Every site is ready to commit if instructed to do so

Three Phase Commit (3PC)

- Phase 2 of 2PC is split into 2 phases, Phase 2 and Phase 3 of 3PC
 - In phase 2 coordinator makes a decision as in 2PC (called the pre-commit decision) and sends pre-commit msg.
Participants respond with acknowledgement message
 - In phase 3, coordinator sends commit/abort message to all participating sites,
- Under 3PC, knowledge of pre-commit decision can be used to commit despite coordinator failure
 - Avoids blocking
- Drawbacks:
 - higher overheads
 - assumptions may not be satisfied in practice



3PC outline (garbage collection not shown)

29

Coordinator:

multicast: *ok to commit?*

collect replies

all *ok* → log "precommit"

multicast *precommit*

else → multicast *abort*

collect *acks* from non-failed participants

all *ack* → log "commit"

multicast *commit*

collect *acks*

garbage-collect protocol outcome info

Participant: logs state on each message

ok-to-commit →

save to temp area, reply *ok*

precommit →

enter precommit state, *acknowledge*

commit →

make change permanent

abort →

delete temp area

After failure:

collect participant state information

any precommit or committed →

push forward to commit

else →

push back to abort

Handling of Failures - Site Failure

When site S_i recovers, it examines its log to determine the fate of transactions active at the time of the failure.

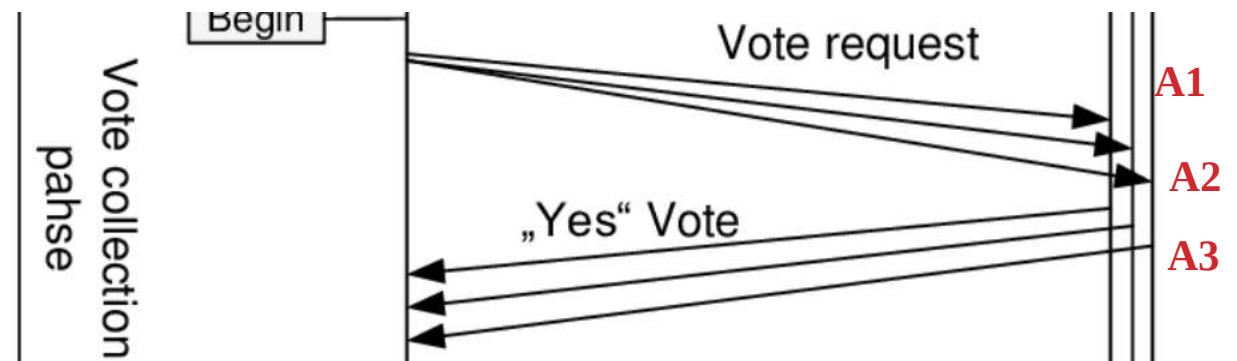
A3: Log contains $\langle \text{ready } T \rangle$ record: site must consult C_i to determine the fate of T .

- If T committed, redo (T)
- If T aborted, undo (T)

A1 A2: (includes (a) After $\langle \text{prepare } T \rangle$ before $\langle \text{ready } t \rangle$ and (b) Before $\langle \text{prepare } T \rangle$)

The log contains no control records concerning T replies that S_k failed before responding to the prepare T message from C_i

- since the failure of S_k precludes the sending of such a response C_i must abort T
- S_k must execute undo (T)

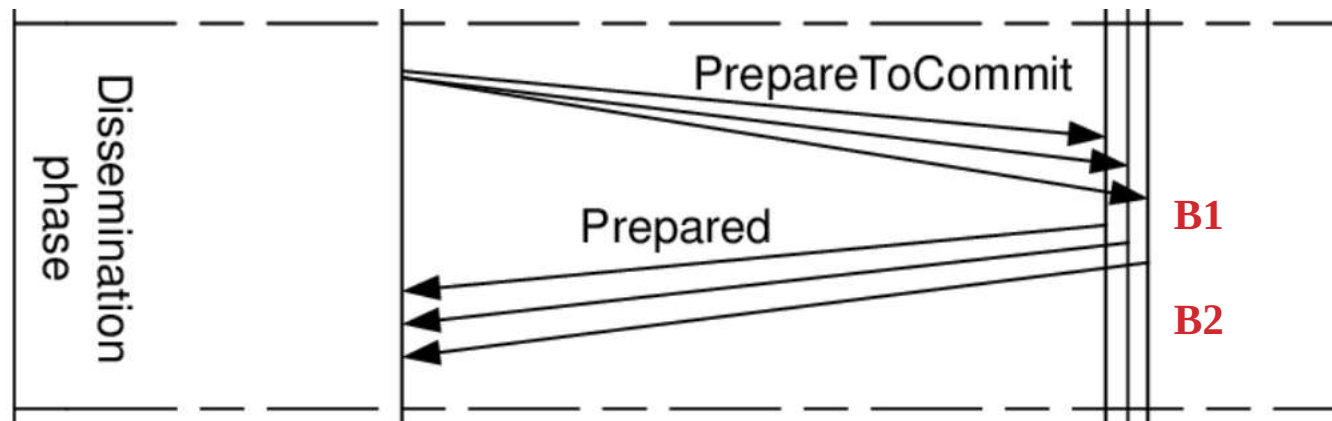


Handling of Failures - Site Failure, continued....

- **B1:** Log contain $\langle \text{pre-commit } T \rangle$ record: site needs to check with coordinator.

The transaction can commit/abort situation (if the coordinator failed after sending precommit to site S_i (S_i also had failed). So remaining sites seeing no precommit msgs among them would abort. OR hearing from all alive sites it can commit)

- **B2:** Log contains $\langle \text{ack } T \rangle$ record: same as B1



Handling of Failures - Site Failure

When site S_i recovers, it examines its log to determine the fate of transactions active at the time of the failure.

- **F:** Log contain $\langle \text{commit } T \rangle$ record: site executes redo (T) **copies from log to db*
- **F:** Log contains $\langle \text{abort } T \rangle$ record: site executes undo (T) **remove from log*



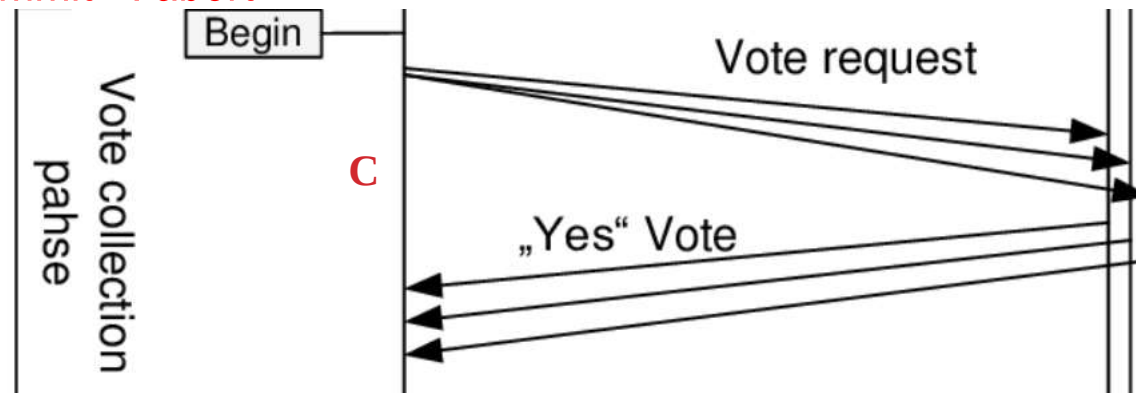
Handling of Failures- Coordinator Failure

- If coordinator fails while the commit protocol for T is executing then participating sites m

C: If some active participating site does not contain a $\langle \text{ready } T \rangle$ record in its log, then the failed coordinator C_i cannot have decided to commit T . Can therefore abort T .

if all contain $\langle \text{ready } T \rangle$ then look for someone who has $\langle \text{preCommit } T \rangle$ to find new coordinator among them

If no one has $\langle \text{precommit} \rangle$: abort



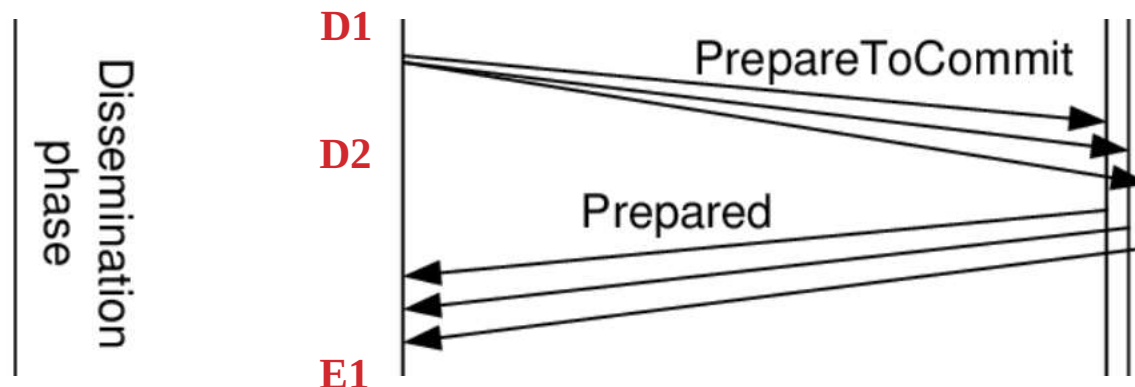
Handling of Failures- Coordinator Failure

- If coordinator fails while the commit protocol for T is executing then participating sites must decide on T 's fate:
 1. all active sites have a $\langle \text{ready } T \rangle$ record in their logs, but no additional control records (such as $\langle \text{abort } T \rangle$ or $\langle \text{commit } T \rangle$).

D2,E1: If there is $\langle \text{precommit } T \rangle$ in someone's log then vote for new coordinator. New coordinator behaves as if it received $\langle \text{ready } T \rangle$ from everyone. It sends $\langle \text{precommit } T \rangle$ to all.

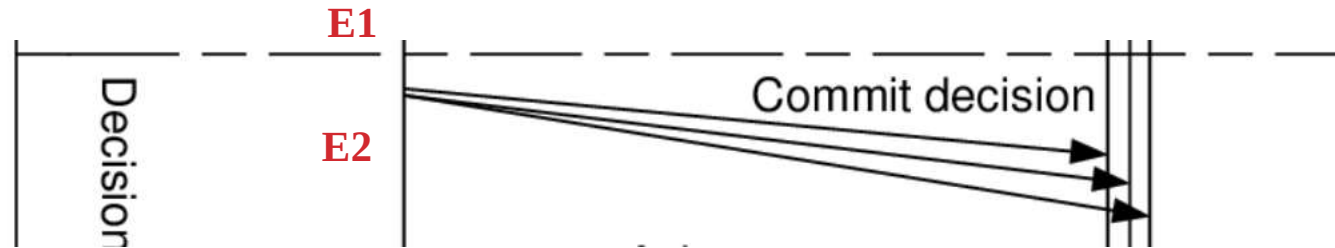
D1: Abort (same as C)

Unlike 2phase this step does not block. In 2PC we had to wait for the coordinator to recover.



Handling of Failures- Coordinator Failure

- If coordinator fails while the commit protocol for T is executing then participating sites must decide on T 's fate:
 1. **E2**: If an active site contains a $\langle \text{commit } T \rangle$ record in its log, then T must be committed.
 2. **E2**: If an active site contains an $\langle \text{abort } T \rangle$ record in its log, then T must be aborted.



Summary – Coordinator Failure

1. Before sending <Vote Request> :
 - Participants time out. State INIT. aborts
2. After sending <Vote Request> & Before receiving votes:
 - Some participants received some did not.
 - Those did not receive time out. State INIT(case 1) & abort.
 - Those who received <Vote Request> will reply and time out & abort
 - All in INIT or Wait with yes/no vote. Aborts.
3. After receiving all <Yes Vote> before Precommit:
 - All participants timeout in Wait state. New coordinator aborts
4. Crash After Sending PreCommit to Some, But Not All:
 - Some received and replied with <ack> and wait and timeout
 - Some did not receive and timeout
 - New Coordinator elected. Gets information from all nodes.
 - If all nodes have precommit then commit
 - If at least one precommit proceeds towards commit by restarting phase 2
 - If no precommit due to nodes with info failing (appears like case 3) then abort.
5. Crash after receiving <acks>: Same as case 4
6. Crash after sending abort/commit: Some receive and some don't.
 - Those who timeout have <precommit> (case 4) – start leader election.
 - Find someone has commit/abort and do same.
 - If no one has abort/commit then elected leader will commit